Лабораторная работа № 5
«Знакомство с библиотекой NumPy»

Цель работы:

Научиться работать с многомерными массивами, в том числе и без явного использования циклов. Получить представление о библиотеке NumPy.

Теоретические сведения

Numeric Python (NumPy)- это несколько модулей для вычислений с многомерными массивами, необходимых для многих численных приложений. Массив - это набор однородных элементов, доступных по индексам. Массивы модуля Numeric могут быть многомерными, то есть иметь более одной размерности. Количество размерностей и длина массива по каждой оси называются формой массива (shape). Размещение массива в памяти проводится в соответствии с опциями и может быть выполнено как в языке C (по последнему индексу), как в языке Fortran (по первому индексу) или беспорядочно.

Особо подчеркнем отличие массива от набора данных (списка или кортежа). Величины, входящие в массив, имеют одинаковый **тип** и их количество жестко задается при инициализации. Элементы массива не являются объектами, это переменные в обычном понимании этого слова. Массивы позволяют экономить память и увеличивать скорость работы с большим количеством однотипных данных.

В Python массивы – это объекты, содержащие буфер данных и информацию о форме, размерности, типе данных и т.д. Как и у любого объекта, у массива можно менять атрибуты напрямую: **array.shape=(2,3)** или через вызов функции **np.reshape(array,(2,3))**. Такая же ситуация и с методами (функциями для массивов), заданными в этом классе, многие из них могут вызываться как методы **array.resize(2,4)** или как самостоятельные функции `NumPy`: **np.resize(array,(2,4))**. Некоторые функции являются только методами: **array.flat**, **array.flatten**, **array.T**. Для правильного использования таких функций, необходимо обращаться к их описанию.

Типы данных

В качестве элементов массива можно использовать множество типов, которые перечислены в таблице.

Типы Int, UnsignedInteger, Float и Complex соответствуют наибольшим принятым на данной платформе значениям.

Тип	Описание	Тип	Описание
Логический		**Числа с плавающей точкой**	
bool_	Python bool	half	
bool8	8 бит	single	C float
Целый		double	C double
byte	C char	float_	Python float
short	C short	longfloat	C long float
intc	C int	float16	16 бит
int_	Python int	float32	32 бит
longlong	C long long	float64	64 бит
intp	такой ,что может описывать указатели	float96	96 бит,???
int8	8 бит	float128	128 бит ???
int16	16 бит	**Комплексные**	
int32	32 бит	csingle	
int64	64 бит	complex_	Python complex
Целый без знака (натуральное)		clongfloat	
ubyte	C unsigned char	complex64	два 32- битовых
ushort	C unsigned short	complex128	два 64- битовых
uintc	C unsigned int	complex192	два 96- битовых ???
uint	Python int	complex256	два 128-битовых
ulonglong	C long long	**Строки**	
uintp	такой, что может описывать указатели	str_	Python str
uint8	8 бит	unicode_	Python unicode
uint16	16 бит	void	
uint32	32 бит	**Объекты Python**	
uint64	64 бит	object_	любой объект Python

Операции для работы с массивами

В NumPy реализовано много операций для работы с массивами:

- создание, модификация массива (изменение формы, транспонирование, поэлементные операции),
- выбор элементов,
- операции с массивами (различные типы умножения), сравнение массивов
- решение задач линейной алгебры (системы линейных уравнений, собственые вектора. собственные значения)
- создание наборов случайных данных
- быстрое преобразование Фурье

Создание массивов

В NumPy можно выделить три вида массивов:

- произвольные многомерные массивы (array)
- матрицы (matrix) – двухмерные квадратные массивы, для которых дополнительно определены операции возведения в степень и перемножения. Для работы с матрицами можно вместо “numpy” подключать “numpy.matrix”, в котором реализованы те же самые операции, только массивы – результаты операций будут приводится к типу “matrix”.
- сетки (grid) – массивы, в которых записаны значения координат точек сети (обычно ортогональной). Сетки позволяют удобно вычислять значение функций многих переменных.

Создание массивов из имеющихся данных

Для создания массивов существует множество функций. Самая распространенная из них **`array()`**.

```
>>> np.array([[1, 2], [3, 4]])
array([[1, 2],
       [3, 4]])
```

Команда	Описание
array(object[, dtype, copy, order, subok, ndmin])	Создать массив
asarray(a[, dtype, order])	Преобразовать в массив
ascontiguousarray(a[, dtype])	Размещает в памяти непрерывный массив(порядок данных как в Си)
asmatrix(data[, dtype])	Представить данные как матрицу
copy(a)	Возвращает копию объекта
frombuffer(buffer[, dtype, count, offset])	Использует буфер, как одномерный массив
fromfile(file[, dtype, count, sep])	Создает массив из данных файла
fromfunction(function, shape, **kwargs)	Создает и заполняет массив значениями функции от индексов элемента
fromiter(iterable, dtype[, count])	Создает одномерный массив из итератора
fromstring(string[, dtype, count, sep])	Создает одномерный массив из строки
loadtxt(fname[, dtype, comments, delimiter, …])	Создает массив из данных текстового файла

- a - объект или массив
- object - любой объект с упорядоченными данными
- dtype - тип данных (если не указан определяется по данным объекта)
- copy - да/нет, создать копию данных
- order - {‘C’, ‘F’, ‘A’}– порядок размещения элементов в памяти (Си, Фортран, любой)

- ndmin - минимальное число измерений (добавляет пустые массивы по недостающим измерениям)
- buffer - объект буфера
- count - число данных для чтения
- offset - отступ от начала
- file - объект файла
- sep - шаг чтения файла
- string - строка
- function - функция. Вызывается function(i,j,k,**kwargs), где i,j,k – индексы ячейки массива
- shape - форма массива
- **kwargs - словарь параметров для функции
- fname – имя файла
- comments – символ коментария
- delimiter – разделитель данных

Создание сеток

Команда	Описание	Пример
arange([start,] stop [, step,][, dtype])	Похоже на “range()”	`>>> np.arange(3.0)` `array([ 0.,  1.,  2.])`
linspace(start, stop [, num, endpoint, retstep])	Равномерный набор точек	`>>> np.linspace(2.0, 3.0, num=5)` `array([ 2., 2.25, 2.5, 2.75, 3.])`
logspace(start, stop [, num, endpoint, base])	Логарифмический набор точек	`>>> np.logspace(2.0, 3.0, num=4, base=2.0)` `array([ 4., 5.03968, 6.34960, 8.])`

meshgrid(x, y)	два вектора, описывающих точки ортогональной сетки.	`>>> X, Y = np.meshgrid([1,2,3], [4,5,7])`
		`>>> X`
		`array([[1, 2, 3],`
		`[1, 2, 3],`
		`[1, 2, 3]])`
		`>>> Y`
		`array([[4, 4, 4],`
		`[5, 5, 5],`
		`[7, 7, 7]])`

mgrid	полный набор данных, описывающий многомерную равномерную ортогональную сетку (X,Y) или (X,Y,Z). Аргументы по каждому измерению: (start : stop : step). Если step	`>>> np.mgrid[0:5:3j,0:5:3j]`
		`array([[[0., 0., 0.],`
		`[2.5, 2.5, 2.5],`
		`[5., 5., 5.]],`
		`[[0., 2.5, 5.],`
		`[0., 2.5, 5.],`

Команда	Описание	Пример
	– мнимое (5j) – то задается количество интервалов разбиения	`[0., 2.5, 5.]]])`

ogrid	сокращенный набор данных, описывающий многомерную равномерную ортогональную сетку (X,Y) или (X,Y,Z). Аргументы по каждому измерению: (start : stop : step). Если step – мнимое (5j) – то задается количество интервалов разбиения	`>>> ogrid[0:5,0:5]`
		`[array([[0],`
		`[1],`
		`[2],`
		`[3],`
		`[4]]), array([[0, 1, 2, 3, 4]])]`

-
- start – начало
- stop – окончание (для «arrange» по умолчанию НЕ включается, для остальных функций - включается)
- step – шаг
- num – число точек в выходном наборе
- endpoint – да/нет, включать крайнюю точку в набор данных
- retstep – да/нет, добавить в данные величину интервала
- x, y – одномерные массивы разбиения для осей.
- base – основание логарифма

Создание массивов определенного вида

Команда	Описание	Пример

empty(shape[, dtype, order]), empty_like(a [, dtype, order, subok])	выделяет место без инициализации (случайные числа)	`>>> np.empty([2, 2], dtype=int)`
		`array([[-1073741821, -1067949133],`
		`[ 496041986, 19249760]])`

eye(N[, M, k, dtype])	двухмерный диагональный со сдвигом	`>>> np.eye(3, k=1)`
		`array([[ 0., 1., 0.],`
		`[ 0., 0., 1.],`
		`[ 0., 0., 0.]])`

identity(N[, dtype])	единичная матрица (квадратная)	`>>> np.identity(3)`
		`array([[ 1., 0., 0.],`
		`[ 0., 1., 0.],`
		`[ 0., 0., 1.]])`

ones(shape[, dtype, order]) ones_like(a[, out])	все единицы	
zeros(shape[, dtype, order]) zeros_like(a[, dtype, order, subok])	все нули	

Команда	Описание	Пример

tri(N[, M, k, dtype])	нижняя треугольная (из единиц)	`>>> np.tri(3, 5, 2, dtype=int)`
		`array([[1, 1, 1, 0, 0],`
		`[1, 1, 1, 1, 0],`
		`[1, 1, 1, 1, 1]])`

tril(a[, k])	вырезание нижней треугольной	`>>>`
		`np.tril([[1,2,3],[4,5,6],[7,8,9],`
		`[10,11,12]], -1)`
		`array([[ 0, 0, 0],`
		`[ 4, 0, 0],`
		`[ 7, 8, 0],`
		`[10, 11, 12]])`

triu(a[, k])	вырезание верхней треугольной	`>>>`
		`np.triu([[1,2,3],[4,5,6],[7,8,9],`
		`[10,11,12]], -1)`
		`array([[ 1, 2, 3],`
		`[ 4, 5, 6],`
		`[ 0, 8, 9],`
		`[ 0, 0, 12]])`

diag(a[, k])	вырезает диагональ или создает двумерную диагональную матрицу	
diagflat(a[, k])	двумерная диагональная матрица со всеми элементами из a.	

vander(x[, N])	создает определительВан Дер Монда	`>>> x = np.array([1, 2, 3, 5])`
		`>>> N = 3`
		`>>> np.vander(x, N)`
		`array([[ 1, 1, 1],`
		`[ 4, 2, 1],`
		`[ 9, 3, 1],`
		`[25, 5, 1]])`

mat(data[, dtype])	преобразует данные в матрицу	
bmat(obj[, ldict, gdict])	создает матрицу из строки, последовательности или массива	

- shape – форма массива
- dtype – тип данных
- order – порядок размещения данных(Си, Фортран)
- a – объект типа массива
- N – число строк
- M – число столбцов
- k – задает диагональ (к=0 – главная, к>0 – смещение вверх, к<0 – смещение вниз)

- x – одномерный массив или список

Трансформации массива без изменения элементов

Команда	Описание
resize(a, new_shape)	возвращает новый массив заданной формы (если элементов не хватает, то заполняется циклически)
reshape(a, newshape[, order])	новая форма для данных (полный размер обязан совпадать)
ravel(a[, order])	возвращает новый одномерный массив
ndarray.flat	итератор по массиву (вызывается как метод)
ndarray.flatten([order])	копия массива без формы (вызывается как метод)
rollaxis(a, axis[, start])	сдвигает выбранную ось до нужного положения
swapaxes(a, axis1, axis2)	меняет две оси в массиве
ndarray.T	То же что и транспонирование. (если размерность=1, то не изменяется)
transpose(a[, axes])	транспонирует массив (переставляет измерения)
fliplr(a)	симметрично отображает массив относительно вертикальной оси (право-лево)
flipud(a)	симметрично отображает массив относительно горизонтальной оси (верх-низ)
roll(a, shift[, axis])	циклический сдвиг элементов вдоль выбранного направления
rot90(a[, k])	поворот массива против часовой стрелке на 90 градусов
tile(A, repeats)	создает матрицу повторением заданной определенное количество
repeat(a, repeats[, axis])	повторяет элементы массива

- a
- newshape
- order
- start
- axis1, axis2
- shift
- k
- repeats

Слияние и разделение массивов

Команда	Описание
column_stack(tup)	собирает одномерные массивы -столбцы в двухмерный
concatenate(tup[, axis])	соединяет последовательность массивов вместе
dstack(tup)	собирает массивы «по глубине» (по третьей оси).
hstack(tup)	собирает массивы «по горизонтали» (по столбцам).
vstack(tup)	собирает массивы «по вертикали» (по строкам).
array_split(a, indices_or_sections[, axis])	разделяет массив по порциям
dsplit(a, indices_or_sections)	разделяет массив «по глубине» (по третьей оси)
hsplit(a, indices_or_sections)	разделяет массив «по горизонтали» (по столбцам).

Команда	Описание
split(a, indices_or_sections[, axis])	разделяет массив на части равной длины
vsplit(a, indices_or_sections)	разделяет массив «по вертикали» (по строкам).

- tup – кортеж массивов
- axis – ось
- a – массив
- indices_or_sections – размер порции при разделении

Функции, определенные для массивов

Алгебраические функции

Функция	Описание
isreal(x)	проверка на действительность (по элементам)
iscomplex(x)	проверка на комплексность (по элементам)
isfinite(x[, out])	приверка элементов на числовое значение (не бесконечность и не «не число»).
isinf(x[, out])	проверка на бесконечность (по элементам)
isnan(x[, out])	проверка аргумента на «не число» (NaN), результат – логический массив
signbit(x[, out])	истина, если установлен бит знака (меньше нуля)
copysign(x1, x2[, out])	меняет знак x1 на знак x2 (по элементам
nextafter(x1, x2[, out])	следующее в направлении x2 число, представимое в виде с плавающей точкой (по элементам)
modf(x[, out1, out2])	дробная и целая часть числа
ldexp(x1, x2[, out])	вычисляет y = x1 * 2**x2.
frexp(x[, out1, out2])	разделение числа на нормированную часть и степень
absolute(x[, out])	Calculate the absolute value element-wise.
rint(x[, out])	округление элементов массива
trunc(x[, out])	отбрасывание дробной части (по элементам)
floor(x[, out])	целая часть
ceil(x[, out])	минимальное целое большее числа
sign(x[, out])	знаки элементов
conj(x[, out])	комплексное сопряжение (по элементам).
exp(x[, out])	экспонента (по элементам)
exp2(x[, out])	2**элемент (по элементам)
log(x[, out])	натуральный логарифм (по элементам)
log2(x[, out])	двоичный логарифм (по элементам)
log10(x[, out])	десятичный логарифм (по элементам)
expm1(x[, out])	exp(x) - 1 (по элементам)
log1p(x[, out])	Return the natural logarithm of one plus the input array, element-wise.
sqrt(x[, out])	квадратный корень (для положительных) (по элементам)
square(x[, out])	квадрат (по элементам)

Функция	Описание
reciprocal(x[, out])	обратная величина (по элементам)

- x – массив
- out – место для результата

Тригонометрические функции

Все тригонометрические функции работают с радианами.

Функция	Обратная функция	Описание
sin(x[, out])	arcsin(x[, out])	синус (по элементам)
cos(x[, out])	arccos(x[, out])	косинус (по элементам)
tan(x[, out])	arctan(x[, out])	тангенс (по элементам)
	arctan2(x1, x2[, out])	арктангенс x1/x2 с правильным выбором четверти (по элементам)
hypot(x1, x2[, out])		гипотенуза по двум катетам (по элементам)
sinh(x[, out])	arcsinh(x[, out])	гиперболический синус (по элементам)
cosh(x[, out])	arccosh(x[, out])	гиперболический косинус (по элементам)
tanh(x[, out])	arctanh(x[, out])	гиперболический тангенс (по элементам)
deg2rad(x[, out])	rad2deg(x[, out])	преобразование градусов в радианы (по элементам)

- x, x1, x2 – массивы
- out – место для результата

Функции двух аргументов (бинарные функции)

Для правильной работы с логическими бинарными функциям (AND, OR) необходимо явно их записывать через функции модуля «NumPy», а не полагаться на встроенные функции питона.

Функция	Описание
add(x1, x2[, out])	сумма (по элементам)
subtract(x1, x2[, out])	разность (по элементам)
multiply(x1, x2[, out])	произведение (по элементам)
divide(x1, x2[, out])	деление (по элементам)
logaddexp(x1, x2[, out])	логарифм суммы экспонент (по элементам)
logaddexp2(x1, x2[, out])	логарифм по основанию 2 от суммы экспонент (по элементам)
true_divide(x1, x2[, out])	истинное деление (с преобразованием типов)
floor_divide(x1, x2[, out])	деление без преобразования типов (целочисленное)
negative(x[, out])	обратные элементы (по элементам)

Функция	Описание
power(x1, x2[, out])	элементы первого массива в степени элементов из второго массива (по элементам)
remainder(x1, x2[, out]), mod(x1, x2[, out]), fmod(x1, x2[, out])	остаток от деления (по элементам).
greater(x1, x2[, out])	истина, если (x1 > x2) (по элементам).
greater_equal(x1, x2[, out])	истина, если (x1 > =x2) (по элементам).
less(x1, x2[, out])	истина, если (x1 < x2) (по элементам).
less_equal(x1, x2[, out])	истина, если (x1 =< x2) (по элементам).
not_equal(x1, x2[, out])	истина, если (x1 != x2) (по элементам).
equal(x1, x2[, out])	истина, если (x1 == x2) (по элементам).
logical_and(x1, x2[, out])	истина, если (x1 AND x2) (по элементам).
logical_or(x1, x2[, out])	истина, если (x1 OR x2) (по элементам).
logical_xor(x1, x2[, out])	истина, если (x1 XOR x2) (по элементам).
logical_not(x[, out])	истина, если (NOT x1) (по элементам).
maximum(x1, x2[, out])	максимум из элементов двух массивов(по элементам).

- x1, x2 – массивы
- out – место для результата

Бинарные функции поддерживают дополнительные методы, позволяющие накапливать значения результата различными способами.

- accumulate() Аккумулирование результата.
- outer() Внешнее «произведение».
- reduce() Сокращение.
- reduceat() Сокращение в заданных точках.

Методы accumulate(), reduce() и reduceat() принимают необязательный аргумент - номер размерности, используемой для соответствующего действия. По умолчанию применяется нулевая размерность.

Другие функций для массивов

Функция	Описание
apply_along_axis(func1d, axis, a, *args)	Применить функцию к одномерному срезу вдоль оси
apply_over_axes(func, a, axes)	применить функцию последовательно вдоль осей.
vectorize(pyfunc[, otypes, doc])	обобщить функцию на массивы
frompyfunc(func, nin, nout)	берет произвольную функцию Python и возвращает функцию Numpy
piecewise(a, condlist, funclist, *args, **kw)	применение кусочно-определенной функции к массиву

- func1d – функция для вектора
- func – скалярная функция
- axis – индекс оси
- arr – массив
- *args, **kw – дополнительные аргументы
- nin – число входных параметров
- nout – число выходных параметров
- condlist - список условий
- funclist – список функций (для каждого условия)

Сортировка, поиск, подсчет

Команда	Описание
sort(a[, axis, kind, order])	отсортированная копия массива
lexsort(keys[, axis])	Perform an indirect sort using a sequence of keys.
argsort(a[, axis, kind, order])	аргументы, которые упорядочивают массив
array.sort([axis, kind, order])	сортирует массив на месте (метод массива)
msort(a)	копия массива отсортированная по первой оси
sort_complex(a)	сортировка комплексного массива по действительной части, потом по мнимой
argmax(a[, axis])	индексы максимальных значений вдоль оси
nanargmax(a[, axis])	индексы максимальных значений вдоль оси (игнорируются NaN).
argmin(a[, axis])	индексы минимальных значений вдоль оси
nanargmin(a[, axis])	индексы минимальных значений вдоль оси (игнорируются NaN).
argwhere(a)	массив индексов ненулевых элементов. данные сгруппированы по элементам([x1,y1,..],[x2,y2,..]….)

Команда	Описание
nonzero(a)	массивы индексов ненулевых элементов. сгруппированы по размерностям (индексы X, индексы Y, т.д.)
flatnonzero(a)	индексы ненулевых элементов в плоской версии массива
where(condition, [x, y])	возвращает массив составленный из элементов x (если выполнено условие) и y (в противном случае). Если задано только condition, то выдает его «не нули».
searchsorted(a, v[, side])	индексы мест, в которые нужно вставить элементы вектора для сохранения упорядоченности массива
extract(condition, a)	возвращает элементы (одномерный массив), по маске (condition)
count_nonzero(a)	число ненулевых элементов в массиве

- a – массив
- axis – индекс оси для сортировки (по умолчанию «–1» - последняя ось)
- kind – {‘quicksort’, ‘mergesort’, ‘heapsort’} тип сортировки
- order – индексы элементов, определяющие порядок сортировки
- keys – (k,N) массив из k элементов размера (N). k “колонок” будут отсортированы. Последний элемент – первичный ключ для сортировки.
- condition – матрица условий (маска)
- x, y – массивы для выбора элементов
- v – вектор
- side – {‘left’, ‘right’} позиция для вставки элемента (слева или справа от найденного индекса)

Дискретное преобразование Фурье (numpy.fft)

Прямое преобразование	Обратное преобразование	Описание
fft(a[, s, axis])	ifft(a[, s, axis])	одномерное дискретное преобразование Фурье
fft2(a[, s, axes])	ifft2(a[, s, axes])	двумерное дискретное преобразование Фурье
fftn(a[, s, axes])	ifftn(a[, s, axes])	многомерное дискретное преобразование Фурье
rfft(a[, s, axis])	irfft(a[, s, axis])	одномерное дискретное преобразование Фурье (действительные числа)
rfft2(a[, s, axes])	irfft2(a[, s, axes])	двумерное дискретное преобразование Фурье (действительные числа)
rfftn(a[, s, axes])	irfftn(a[, s, axes])	многомерное дискретное преобразование Фурье (действительные числа)
hfft(a[, s, axis])	ihfft(a[, s, axis])	преобразование Фурье сигнала с Эрмитовым спектром
fftfreq(n[, d])		частоты дискретного преобразования Фурье
fftshift(a[, axes])	ifftshift(a[, axes])	преобразование Фурье со сдвигом нулевой компоненты в центр спектра

- a - массив

- s – число элементов вдоль каждого направления преобразования (если больше размерности, то дополняются нулями)
- axes – последовательность осей для преобразования
- n – ширина окна
- d – шаг по частоте при выводе

Линейная алгебра (numpy.linalg)

Модуль numpy.linalg содержит алгоритмы линейной алгебры, в частности нахождение определителя матрицы, решений системы линейных уравнений, обращение матрицы, нахождение собственных чисел и собственных векторов матрицы, разложение матрицы на множители: Холецкого, сингулярное, метод наименьших квадратов и т.д.

Команда	**Описание**
dot(a, b[, out])	скалярное произведение массивов
vdot(a, b)	векторное произведение векторов
inner(a, b)	внутреннее произведение массивов
outer(a, b)	внешнее произведение векторов
tensordot(a, b[, axes])	тензорное скалярное произведение вдоль оси (размерность больше 1)
einsum(subscripts, *operands[, out, dtype, …])	суммирование Эйншнейна Evaluates the Einstein summation convention on the operands.
linalg.matrix_power(M, n)	возведение квадратной матрицы в степень n
kron(a, b)	произведение Кронекера двух массивов
linalg.norm(a[, ord])	норма матрицы или вектора.
linalg.cond(a[, ord])	число обусловленности матрицы.
linalg.det(a)	определитель
linalg.slogdet(a)	знак и натуральный логарифм определителя
trace(a[, offset, axis1, axis2, dtype, out])	сумма элементов по диагонали.
linalg.cholesky(a)	разложение Холецкого
linalg.qr(a[, mode])	разложение QR
linalg.svd(a[, full_matrices, compute_uv])	сингулярное разложение
linalg.solve(a, b)	решение линейного матричного уравнения или системы скалярных уравнений.
linalg.tensorsolve(a, b[, axes])	решение тензорного уравнения a x = b для x.
linalg.lstsq(a, b[, rcond])	решение матричного уравнения методом наименьших квадратов
linalg.inv(a)	обратная матрица (для умножения)
linalg.pinv(a[, rcond])	псевдо-обратная матрица (Мура-Пенроуза)
linalg.tensorinv(a[, ind])	«обратный» к многомерному массиву
linalg.eig(a)	собственные значения и правые собственные вектора квадратной
linalg.eigh(a[, UPLO])	собственные значения и собственные вектора эрмитовой или симметричной матрицы

Команда	Описание
linalg.eigvals(a)	собственные значения произвольной матрицы
linalg.eigvalsh(a[, UPLO])	собственные значения эрмитовой или действительной симметричной матрицы

- a, b - матрицы
- out - место для результата
- ord – определяет способ вычисления нормы
- axes – массив осей для суммирования
- axis – индекс оси
- subscripts – индексы для суммирования
- *operands – список массивов
- dtype – тип результата
- offset – положение диагонали
- mode - {‘full’, ‘r’, ‘economic’} – выбор алгоритма разложения
- full_matrices – составлять полные матрицы
- compute_uv – выводить все матрицы
- rcond – граница для отбрасывания маленьких собственных значений
- ind – число индексов для вычисления обратной
- UPLO - {‘L’, ‘U’} выбирает часть матрицы для работы

Случайные величины (numpy.random)

В модуле numpy.random собраны функции для генерации массивов случайных чисел различных распределений и свойств. Их можно применять для математического моделирования. Функция random() создает массивы из псевдослучайных чисел, равномерно распределенных в интервале (0, 1). Функция RandomArray.randint() для получения массива равномерно распределенных чисел из заданного интервала и заданной формы. Можно получать и случайные перестановки с помощью RandomArray.permutation(). Доступны и другие распределения для получения массива нормально распределенных величин с заданным средним и стандартным отклонением:

Следующая таблица приводит основные функции модуля.

seed([seed])- перезапуск генератора случайных чисел

Команда	Описание
rand(d0, d1, …, dn)	набор случайных чисел заданной формы
randn([d1, …, dn])	набор (или наборы) случайных чисел со стандартным нормальным распределением
randint(low[, high, size])	случайные целые числа от low (включая) до high (не включая).
random_integers(low[, high, size])	случайные целые числа между low и high (включая).

Команда	Описание
random_sample ([size])	случайные рациональные числа из интервала [0.0, 1.0).
bytes(length)	случайные байты
shuffle(x)	тасовка элементов последовательностина месте
permutation(x)	возвращает последовательность, переставленных случайным образом элементов
seed([seed])	перезапуск генератора случайных чисел
beta(a, b[, size])	числа с Бетта-распределением $f(x,\alpha,\beta)=1B(\alpha,\beta)x\alpha-1(1-x)\beta-1f(x,\alpha,\beta)=1B(\alpha,\beta)x\alpha-1(1-x)\beta-1$ $B(\alpha,\beta)=\int10t\alpha-1(1-t)\beta-1dtB(\alpha,\beta)=\int01t\alpha-1(1-t)\beta-1dt$
binomial(n, p[, size])	числа с биномиальным распределением $P(N)=(nN)pN(1-p)n-NP(N)=(nN)pN(1-p)n-N$
chisquare(df[, size])	числа с распределением хи-квадрат $p(x)=(1/2)k/2\Gamma(k/2)xk/2-1e-x/2p(x)=(1/2)k/2\Gamma(k/2)xk/2-1e-x/2$
mtrand.dirichlet (alpha[, size])	числа с распределением Дирихле (alpha – массив параметров).
exponential([scale, size])	числа с экспоненциальным распределением $f(x,1\beta)=1\beta exp(-x\beta)f(x,1\beta)=1\beta exp(-x\beta)$
f(dfnum, dfden[, size])	числа с F распределением (dfnum – число степеней свободы числителя > 0; dfden –число степеней свободы знаменателя >0.)
gamma(shape[, scale, size])	числа с Гамма - распределением
geometric(p[, size])	числа с геометрическим распределением
gumbel([loc, scale, size])	числа с распределением Гумбеля
hypergeometric (ngood, nbad, nsample[, size])	числа с гипергеометрическим распределением (n = ngood, m = nbad, and N = number of samples)
laplace([loc, scale, size])	числа с распределением Лапласа
logistic([loc, scale, size])	числа с логистическим распределением
lognormal([mean, sigma, size])	числа с логарифмическим нормальным распределением
logseries(p[, size])	числа с распределением логарифмического ряда
multinomial(n, pvals[, size])	числа с мультиномиальным распределением
multivariate_normal(mean, cov[, size])	числа с мульти нормальным распределением (mean – одномерный массив средних значений; cov – двухмерный симметричный, полож. определенный массив (N, N) ковариаций
negative_binomial(n, p[, size])	числа с отрицательным биномиальным распределением

Команда	Описание
noncentral_chisquare(df, nonc[, size])	числа с нецентральным распределением хи-квадрат
noncentral_f(dfnum, dfden, nonc[, size])	числа с нецентральным F распределением (dfnum - целое > 1; dfden – целое > 1; nonc : действительное >= 0)
normal([loc, scale, size])	числа с нормальным распределением
pareto(a[, size])	числа с распределением Паретто
poisson([lam, size])	числа с распределением Пуассона
power(a[, size])	числа со степенным распределением [0, 1]
rayleigh([scale, size])	числа с распределением Релея
standard_cauchy([size])	числа со стандартным распределением Коши
standard_exponential([size])	числа со стандартным экспоненциальным распределением
standard_gamma(shape[, size])	числа с гамма- распределением
standard_normal([size])	числа со стандартным нормальным распределением (среднее=0, сигма=1).
standard_t(df[, size])	числа со стандартным распределением Стьюдента с df степенями свободы
triangular(left, mode, right[, size])	числа из треугольного распределения
uniform([low, high, size])	числа с равномерным распределением
vonmises(mu, kappa[, size])	числа с распределением Майсеса (I- модифицированная функция Бесселя)
wald(mean, scale[, size])	числа с распределением Вальда
weibull(a[, size])	числа с распределением Вайбулла
zipf(a[, size])	числа с распределением Зипфа (зетта функция Римана)

- size - число элементов по каждому измерению

Статистика

Команда	Описание
amin(a[, axis, out])	минимум в массиве или минимумы вдоль одной из осей
amax(a[, axis, out])	максимум в массиве или максимумы вдоль одной из осей
nanmax(a[, axis])	максимум в массиве или максимумы вдоль одной из осей (игнорируются NaN).

Команда	Описание
nanmin(a[, axis])	минимум в массиве или минимумы вдоль одной из осей (игнорируются NaN).
ptp(a[, axis, out])	диапазон значений (максимум - минимум) вдоль оси
average(a[, axis, weights, returned])	взвешенное среднее вдоль оси
mean(a[, axis, dtype, out])	арифметическое среднее вдоль оси
median(a[, axis, out, overwrite_input])	вычисление медианы вдоль оси
std(a[, axis, dtype, out, ddof])	стандартное отклонение вдоль оси
corrcoef(x[, y, rowvar, bias, ddof])	коэффициенты корреляции
correlate(a, v[, mode, old_behavior])	кросс-корреляция двух одномерных последовательностей
cov(m[, y, rowvar, bias, ddof])	ковариационная матрица для данных
histogram(a[, bins, range, normed, weights, …])	гистограмма из набора данных
histogram2d(x, y[, bins, range, normed, weights])	двумерная гистограмма для двух наборов данных
histogramdd(sample[, bins, range, normed, …])	многомерная гистограмма для данных
bincount(x[, weights, minlength])	число появление значения в массиве неотрицательных значений
digitize(x, bins)	возвращает индексы интервалов к которым принадлежат элементы массива

- a – массив
- axis – индекс оси
- out – место для результата
- weights- веса
- returned – дополнительно выдать сумму весов
- dtype – тип данных для накопления суммы
- overwrite_input – использовать входящий массив для промежуточных вычислений
- ddof- дельта степеней свободы (см. описание)
- x,y – данные (строки – переменные, колонки - наблюдения)
- rowvar – если не 0, то строка переменные, колонки – наблюдения (если 0, то наоборот)
- bias – определеяет нормировку, совместно с ddof
- mode - {‘valid’, ‘same’, ‘full’} – объем выводимых данных
- old_behavior – совместимость со старой версией (без комплексного сопряжения)
- bins – разбиение по интервалам
- range – массив границ по x и по y
- normed – нормированное
- minlength – минимальное число интервалов при выводе

По материалам https://physics.susu.ru/vorontsov/language/numpy.html

Задание на л/р

Каждый студент выполняет все задания!

Последовательно выполните задания:

1. Найти индексы ненулевых элементов в заданном массиве.
2. Создать nxn единичную матрицу.
3. Создать массив nxnxn со случайными значениями
4. Создать вектор (одномерный массив) размера n, заполненный нулями
5. Создать вектор размера 1n, заполненный нулями, но пятый элемент равен 1
6. Создать вектор со значениями от n до m
7. Развернуть одномерный массив (первый становится последним)
8. Создать матрицу с 0 внутри, и 1 на границах
9. Создать nxn матрицу с 1,2,..,n-1 под диагональю
10. Создать nxn матрицу и заполнить её в шахматном порядке
11. Заменить максимальный элемент на ноль
12. Найти ближайшее к заданному значению число в заданном массиве
13. Дан четырехмерный массив, посчитать сумму по последним двум осям
14. Определить, есть ли в 2D массиве нулевые столбцы
15. Отнять среднее из каждой строки в матрице